

WebSentinel: An Explainable Website Security Assessment Framework for Phishing Risk Analysis

Seethina Jaya Dileep*, Ganesh Maheshwaram*, and N V Sai Gokul*

Somineni Venumadhava*, Ruthwik Kakumani*, and Ganta Sai Mani Chand*

*Department and university affiliation to be completed for the camera-ready version.

Corresponding author: jayadileepseethina@gmail.com

Abstract—Non-technical users regularly receive website addresses through electronic mail, messaging applications, advertisements, QR codes, and social media, yet they generally cannot determine whether a destination is legitimate, newly registered, misconfigured, or associated with phishing. Publicly observable evidence—domain age, certificate status, DNS configuration, redirect behavior, and reputation—exists, but it is fragmented across services and is rarely returned as an accountable assessment. HTTPS alone does not establish trustworthiness, because impersonation sites frequently obtain valid certificates. This paper presents WebSentinel, an explainable multi-signal framework for website security and phishing-risk assessment. The proposed architecture aggregates passive public indicators, explains each fired warning, and maps them to an advisory score. The implemented prototype is a browser extension and REST service that transmits only the site origin, inspects HTTPS and TLS, estimates domain age from registration data, and applies a documented rule-based score. Proposed extensions include DNS, RDAP, redirect and technology collectors, approved reputation sources, and an optional machine-learning probability with feature attributions. A further contribution is a scanner threat model: user-controlled URL fetches can become server-side request-forgery paths into private or cloud-metadata networks. No trained-classifier metrics or large-scale scan statistics are reported, because those experiments have not yet been executed. The paper therefore contributes a specified, ethically constrained assessment architecture, a working rule-based prototype, and a reproducible evaluation protocol.

Index Terms—Phishing risk assessment, website security, cybersecurity, explainable artificial intelligence, URL analysis, threat intelligence, server-side request forgery, web security.

I. INTRODUCTION

Phishing remains one of the most effective social-engineering attacks on the public web. Attackers impersonate banks, payment processors, cloud consoles, and social-network login pages in order to harvest credentials or payment data [1], [2]. The Anti-Phishing Working Group continues to document large quarterly volumes of phishing activity, and the lifetime of many campaigns is short enough that purely reactive blacklists lag behind newly registered hosts [2], [3]. Browser security indicators help some users, but a substantial fraction of people do not inspect the address bar or certificate interface, and visual deception can still succeed [1].

A. Background

Public information about a website is abundant. The URL encodes scheme, hostname, and path; HTTP describes status

codes and redirects; and DNS associates names with addresses. Registration data—historically WHOIS and increasingly RDAP—expose creation dates, registrars, and name-servers [4]. TLS certificates describe issuer, validity interval, and subject binding [5]. Threat-intelligence services maintain reputation for known-bad hosts. Each of these channels has been used, separately, in prior anti-phishing research [6]–[9]. Ordinary users, however, cannot query and interpret these sources under time pressure.

B. Problem Statement

The operational problem is therefore not the absence of security signals. It is the combination of (i) fragmented evidence, (ii) a user population that cannot interpret raw WHOIS or certificate fields, and (iii) phishing sites that imitate legitimate brands and often present a padlock icon. A URL received in SMS, email, or a QR code may be a look-alike domain, a newly registered host, an open redirect, or a page served over HTTPS from otherwise suspicious infrastructure. Existing commercial toolbars and Safe Browsing-style classifiers are effective at scale [10] but typically return an opaque block-or-allow decision. Academic detectors that rely only on lexical URL features, only on page content, or only on search-engine queries each miss complementary evidence [11], [12].

WebSentinel addresses this gap by aggregating several publicly observable indicators into one assessment and by explaining *why* a site appears comparatively safe or comparatively risky. The system is intended as an advisory assistant. A low score is not a proof of safety; a high score is a request for caution.

C. Research Motivation

University-capstone and small-organization settings rarely operate a full security-operations pipeline, yet their users still click unknown links. A system that performs only passive or otherwise approved checks against public information—and that refuses to exploit, credential-harvest, or attack the target—is an appropriate response. The same URL-fetching capability can become a vulnerability if the scanner is induced to contact loopback, RFC 1918, or cloud-metadata addresses [13], [14]. Designing the scanner so that it cannot be turned into an internal probe is therefore part of the research problem, not an afterthought.

D. Objectives

The work pursues the following objectives.

- 1) Aggregate important website-security information in a single system.
- 2) Identify common phishing and suspicious-website indicators.
- 3) Analyze HTTP, DNS, RDAP or domain-registration, IP, TLS, redirect, and technology signals, to the extent each collector is implemented.
- 4) Generate an explainable advisory score and explain individual warnings in language suitable for non-technical users.
- 5) Specify, but not prematurely claim, later support for scan history, downloadable reports, machine-learning prediction, approved threat-intelligence sources, and change monitoring.
- 6) Maintain safe and ethical assessment practices, including SSRF-aware URL handling.

E. Main Contributions

This paper makes five contributions.

- 1) An explainable multi-signal architecture for website security and phishing-risk assessment.
- 2) A rule-based prototype that integrates HTTPS, TLS, and domain-age indicators and returns human-readable reasons.
- 3) A scanner threat model that treats SSRF as a first-class risk of user-controlled URL fetching [14], [15].
- 4) An extension architecture for DNS, RDAP, reputation, and an optional machine-learning probability.
- 5) A reproducible evaluation protocol that can be executed once licensed datasets and tests exist; no fabricated numerical results are reported.

The remainder of the paper is organized as follows. Section II reviews related work. Section III states requirements and the threat model. Section IV describes the proposed architecture and the implemented subset. Section V presents the methodology. Section VI discusses implementation choices. Section VII specifies the evaluation protocol. Section VIII reports observations supported by source inspection. Sections IX–XI address privacy, limitations, and future work. Section XII concludes.

II. RELATED WORK

Phishing detection research has evolved from content heuristics and blacklists toward lexical learning, hybrid feature models, visual brand matching, and explainable decision support. This section reviews those lines and positions WebSentinel conceptually against them. Table I summarizes the comparison.

A. User Studies and the Limits of Browser Cues

Dhamija *et al.* showed that many users ignore certificate and address-bar cues and that visual spoofing remains effective even against comparatively sophisticated participants [1]. That finding motivates systems that translate technical evidence into

short reasons rather than assuming that a padlock icon is sufficient.

B. Content-Based and Search-Based Detectors

CANTINA applied TF-IDF robust hyperlinks to decide whether page terms are consistent with a legitimate site, optionally combined with heuristics [6]. CANTINA+ expanded the feature set with DOM, search-engine, and third-party attributes and added filters for near-duplicate phishing pages and login-form absence [8]. PhishStorm computed intra-URL relatedness from search-query data and produced a phishingness rating suitable for streaming deployment [3]. These systems demonstrated that content and search context can catch pages that lexical rules miss, at the cost of search-API dependence and page-download overhead. WebSentinel's prototype does not fetch page HTML for classification; content and technology analysis remain planned.

C. URL and Host Feature Learning

Ma *et al.* showed that lexical and host features can classify malicious URLs without downloading page bodies [7]. Subsequent work engineered richer URL tokenizations and NLP-style features. Sahingoz *et al.* compared seven classifiers on a large URL corpus and reported strong Random Forest performance with language-independent NLP features [11]. Hybrid models combine address-bar features with hyperlink or third-party attributes [9]. Hannousse and Yahiouche argued for reproducible feature-rich benchmarks and released a balanced dataset with URL, content, and external-service features [12].

These studies consistently find that accuracy alone is an incomplete metric: false negatives expose users to credential theft, while false positives erode trust. They also show that external features (registration age, DNS, reputation) are often highly discriminative but introduce latency and third-party failure modes [12]. WebSentinel follows that hybrid philosophy while insisting that every fired indicator be explainable to a non-expert.

D. Reputation, Blacklists, and Browser Protection

Industry detectors such as the large-scale classifier described by Whittaker *et al.* combine crawling, content features, and continually updated labels [10]. Blacklists are effective for known campaigns and weak against zero-hour hosts [2], [3]. WebSentinel treats approved reputation lookups as an optional planned signal, not as the sole decision source.

E. Visual Similarity and Brand Impersonation

Phishpedia identifies brand logos and matches them to claimed identities, producing an explanation in the form of a target brand [16]. Visual brand matching is complementary to WebSentinel's current host-and-certificate focus and is listed as future work rather than an implemented module.

F. Explainable Machine Learning in Security

SHAP assigns consistent additive feature attributions grounded in Shapley values [17]. Phishing tools that emit only a probability leave users unable to judge whether HTTPS, domain age, or a lexical oddity dominated the decision. The proposed design therefore treats SHAP, or an equivalent attribution method, as the explanation layer for any later learned model. The current prototype explains decisions with deterministic rule texts, which are a form of inherent interpretability.

G. Standards Relevant to Collectors and Scanner Safety

TLS 1.3 constrains what a passive scanner may observe on port 443 [5]. RDAP specifies HTTP/JSON access to registration data as a successor to WHOIS [4]. The private-address registry enumerates destinations that a URL scanner must not fetch [13]. OWASP documents SSRF as a systemic web risk and publishes a prevention cheat sheet [14], [15].

H. Positioning

Relative to prior work, WebSentinel is not a new deep classifier and does not claim visual brand matching. Its intended niche is a unified, user-explainable assessment that (i) already ships a conservative rule engine on HTTPS, certificates, and domain age, and (ii) is specified to grow toward DNS, redirects, technologies, reputation, and a documented learning pipeline without converting the scanner into an attack tool. Table I separates the inspected prototype from the proposed system.

III. SYSTEM REQUIREMENTS AND THREAT MODEL

A. Intended Users

The primary user is a non-expert who received a link and wants a short, reasoned caution level before entering credentials. Secondary users are student analysts and small-team operators who may later need stored assessments and a downloadable report. The system is not a penetration-testing platform and is not authorized to assess systems that the user does not have a legitimate reason to inspect as a public client.

B. Assets

Protected assets are (i) the user’s credentials and session tokens, which must never be requested or stored; (ii) the confidentiality of the submitted URL, of which the prototype already minimizes path and query; and (iii) the scanner host itself, including any cloud-metadata credentials reachable via SSRF.

C. Attacker Model

The external attacker crafts URLs and websites intended to impersonate trusted brands. The attacker may register a young domain, obtain a valid publicly trusted certificate, use many subdomains, employ redirects, copy HTML and logos, and rotate hosting. The attacker may also try to cause false negatives by presenting HTTPS and a plausible brand, or

false positives against WebSentinel by mimicking aged, well-configured infrastructure. A second adversary is a user of the scanner—or a page that tricks a user into scanning a URL—who attempts to make the backend fetch internal resources. That adversary is modeled as an SSRF attacker [14].

D. Phishing and Malicious-URL Threat

Phishing pages seek credentials, session cookies, cryptocurrency, or malware delivery. They are often short-lived [2], [12]. Malicious URLs more broadly include drive-by exploit hosts and spam landing pages [7]. WebSentinel’s current rules target *suspiciousness indicators* correlated with phishing and poor operational hygiene; they do not prove criminal intent.

E. Ethical Scanning Boundaries

The system must not steal credentials, attempt exploitation, perform destructive testing, bypass authentication, or conduct unauthorized attacks. Collectors are restricted to information a normal web client or public directory service can observe: scheme, certificate on port 443, and registration records. Future HTML fetches, if enabled, must remain unauthenticated GET requests with strict timeouts and size limits. Approved threat-intelligence APIs may be queried only under their terms of use.

F. System Limitations as Requirements

The service cannot see authenticated application state, cannot execute a full browser DOM without a planned headless client, and cannot guarantee completeness of WHOIS or RDAP fields. Those limits are accepted: the output is advisory.

G. SSRF Threat Against the Scanner

If a user can supply `http://127.0.0.1/`, `http://169.254.169.254/`, an RFC 1918 literal, a DNS name that resolves to such an address, or a redirect that later targets one, the scanner becomes an internal HTTP proxy [13], [14]. Cloud instance-metadata endpoints are a documented high-value target. The control set is specified in Section V-H. The inspected prototype validates scheme and strips path, but it does not yet enforce private-address denial after DNS resolution. That gap is treated as implementation debt, not as a completed control.

IV. PROPOSED SYSTEM ARCHITECTURE

Fig. 1 depicts the proposed architecture. Solid boxes are present in the inspected prototype. Dashed boxes are planned. The current system occupies a strict subset of the figure: a Manifest V3 Chrome popup, an Express.js API, and three collectors (protocol, TLS, and WHOIS).

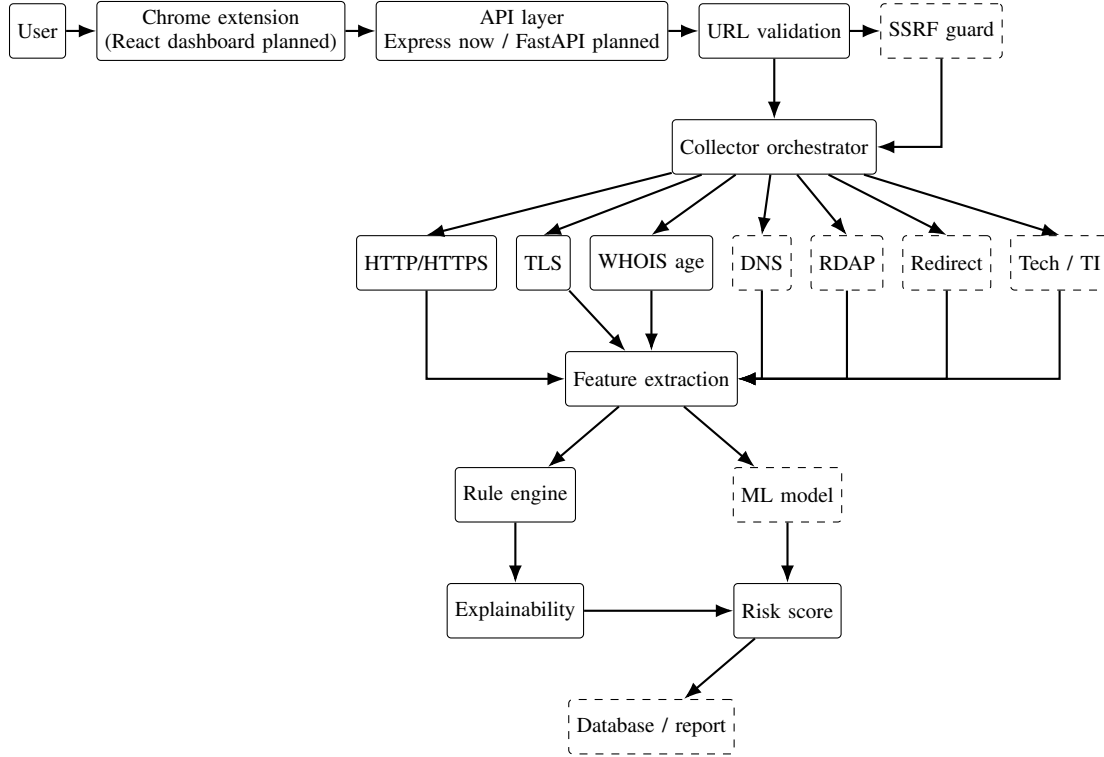
A. Frontend

The proposed user interface is a React, TypeScript, and Tailwind CSS dashboard with URL entry, scan history, and report download. The implemented interface is a Chrome extension popup that reads the active tab, rejects non-HTTP(S) schemes locally, posts only the origin, and renders a verdict (Safe, Suspicious, or High risk), a numeric score, and

TABLE I

CONCEPTUAL COMPARISON OF WEBSentinel WITH REPRESENTATIVE PRIOR APPROACHES. Y: PRIMARY; P: PARTIAL; L: LIMITED; -: NOT A PRIMARY CONTRIBUTION. TI: THREAT INTELLIGENCE; AGE: DOMAIN AGE; XAI: EXPLANATION; SCORE: UNIFIED ADVISORY SCORE. PROTOTYPE VERSUS PROPOSED ROWS REFLECT INSPECTED CODE.

Approach	URL	DNS	TLS	Age	TI	ML	XAI	Score
CANTINA [6]	P	-	-	P	P	-	L	-
CANTINA+ [8]	Y	P	-	P	Y	Y	L	-
PhishStorm [3]	Y	-	-	-	P	Y	P	P
URL ML [11]	Y	-	-	-	-	Y	L	-
Hybrid ML [9]	Y	P	P	P	P	Y	L	-
Safe Browsing-style [10]	Y	-	-	-	Y	Y	-	-
Visual [16]	-	-	-	-	-	Y	P	-
WebSentinel prototype	P	-	Y	Y	-	-	Y	Y
WebSentinel proposed	Y	Y	Y	Y	Y	Y	Y	Y



Legend: solid = implemented prototype; dashed = proposed/planned.

Fig. 1. WebSentinel system architecture. Solid borders denote modules verified in the inspected prototype. Dashed borders denote planned collectors, learning, storage, and SSRF hardening.

a reason list. The extension requests `activeTab` and a single API host permission.

B. Backend

The proposed backend is an asynchronous FastAPI service that orchestrates collectors and the risk engine. The implemented backend is Express.js exposing `GET /health` and `POST /check`, with an 8 kB JSON body limit and CORS enabled. Local listening is bound to `127.0.0.1` unless deployed on Vercel.

C. Data-Collection Modules

Proposed collectors cover HTTP responses, DNS, RDAP, TLS, redirects, IP/hosting/geolocation, HTML technology fingerprints, and optional reputation APIs. Implemented collectors are scheme inspection, TLS inspection via `ssl-checker`, and domain-registration scraping of a public WHOIS HTML endpoint. A file `rdap.js` exists but is empty; RDAP is therefore planned, not implemented.

D. Database and Background Processing

The proposed schema includes users, scans, websites, domain records, DNS records, TLS records, redirects, technologies, risk findings, ML predictions, reports, and monitoring configurations. The inspected prototype persists nothing: each check is stateless. No PostgreSQL models were found. Celery or RQ with Redis is specified for long scans and monitoring. The prototype performs collectors inline with an 8 s per-collector timeout.

E. Machine-Learning Module and Reports

A scikit-learn classifier with optional SHAP explanations is planned. No model artifact, training notebook, or feature-store code was found. Downloadable reports and a React report view are planned. Approved breach or reputation APIs are planned and are not called in the inspected code.

V. METHODOLOGY

A. URL Validation

Let u be the raw input. The prototype parses u with the WHATWG URL parser, optionally prefixing `https://`. A request is rejected if the scheme is not `http` or `https`, or if it matches an unsupported family such as `chrome`, `edge`, `about`, `devtools`, `file`, `data`, or `blob`. The API then discards path, query, and fragment and retains only

$$\text{origin}(u) = \text{scheme}(u) || : // || \text{host}(u). \quad (1)$$

This is a privacy control as well as a validation step: secrets encoded in query strings never leave the browser. Planned validation additionally resolves $\text{host}(u)$ and refuses private, loopback, link-local, and metadata addresses (Section V-H).

B. HTTP, DNS, Domain, and TLS Analysis

The prototype does not issue a full HTTP GET to the target for header or body analysis. It only tests whether the scheme is HTTPS. Planned HTTP analysis will record status code, selected security headers, and a size-limited title, still without authentication.

Independent DNS collection is planned. A resolver would collect A, AAAA, MX, NS, and TXT records. The prototype currently surfaces nameservers only when the WHOIS scrape includes them.

RDAP is the standards-based successor to WHOIS [4]. The prototype instead HTTP-GETs a third-party WHOIS presentation page and parses embedded JSON. Extracted fields include registrar, creation, update, and expiration dates, nameservers, and a DNSSEC string. Domain age is computed from the creation timestamp. This collector is a functional stand-in for RDAP and inherits HTML-structure brittleness.

The prototype connects to port 443 with an 8 s timeout and records validity, validity interval, days remaining, and issuer [5]. An exception is treated as an invalid certificate. HTTPS remains insufficient as a trust signal [1], [2].

C. Redirect, Technology, and Reputation Analysis

Redirect analysis, technology detection, and reputation lookups are planned. A bounded client should follow at most N redirects, re-validate each hop against the SSRF policy, and flag cross-registrable-domain hops. Header and HTML signatures would be collected with a size-capped parser. Reputation may be queried only through approved APIs; a positive hit is an indicator, not a legal determination.

D. Feature Extraction

Let $\mathbf{x}$ be the feature vector. The prototype uses a small discrete set: HTTPS flag, certificate validity, days-to-expiry, domain-age bucket, WHOIS-update recency, and a DNSSEC string. The proposed vector additionally includes URL length, dot and subdomain counts, digit and special-character counts, IP-literal host, redirect count, lexical keywords, login-form cues, and reputation. Features should be computed from already-collected artifacts to avoid extra network risk.

E. Rule-Based Risk Scoring

Two scoring presentations are distinguished: the implemented integer S and a future normalized score R .

a) *Implemented prototype.*: Let S be an integer initialized to 0. Penalties observed in the prototype scoring module are

$S += 20$	if the scheme is not HTTPS,
$S += 20$	if the certificate is missing or invalid,
$S += 3$	if $0 \leq d_{\text{exp}} < 7$,
$S += a(y, m, d)$	age bucket,
$S += u(\Delta)$	WHOIS update recency,
$S -= 3$	if HTTPS and DNSSEC is signed.

The age function a is mutually exclusive:

$$a = \begin{cases} 35, & y = m = 0, d < 7, \\ 25, & y = 0, m < 1, \\ 15, & y = 0, m < 6, \\ 8, & y < 1, \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

Update recency $u(\Delta)$ is 10 if $\Delta < 7$ days, 6 if $\Delta < 30$, else 0. The implementation then applies $S \leftarrow \max(S, 0)$. Verdicts are

$$\begin{cases} \text{Safe,} & S < 24, \\ \text{Suspicious,} & 24 \leq S < 59, \\ \text{High risk,} & S \geq 59. \end{cases} \quad (3)$$

These weights and thresholds are implementation design choices and have not yet been statistically calibrated against a labelled phishing corpus.

The user interface presents S on a 0–100 scale. Under the current mutually exclusive age buckets, the theoretical maximum is $20 + 20 + 35 + 10 = 85$ (HTTP, invalid TLS, age under 7 days, and a WHOIS update under 7 days). The displayed denominator 100 is therefore a presentation scale,

not a reachable sum of the current rule set. The prototype does not implement min–max normalization.

b) *Proposed normalized model.*: For a planned multi-signal engine, let $f_i \in [0, 1]$ be a normalized risk contribution of indicator i and $w_i \geq 0$ its weight. An optional learned probability $p \in [0, 1]$ may be included with weight λ :

$$R_{\text{raw}} = \sum_{i=1}^n w_i f_i + \lambda p, \quad (4)$$

$$R = \text{clip}_{[0,100]} \left(100 \cdot \frac{R_{\text{raw}}}{\sum_i w_i + \lambda} \right). \quad (5)$$

The current prototype score S and the future normalized R are distinct. Normalization is not implemented unless a later revision of the scoring module proves otherwise. A five-level interpretation of R may later be evaluated; those bands are not the bands running in the prototype and must not be presented as validated.

F. Machine-Learning Classification

No classifier is trained in the inspected repository. The planned pipeline, shown in Fig. 4, is: obtain an approved dataset such as the Hannousse–Yahiouche benchmark [12] or a similarly licensed PhishTank/OpenPhish plus legitimate crawl; clean and deduplicate; engineer $\mathbf{x}$; apply a stratified or chronological train/test split; fit candidate models (logistic regression, decision tree, random forest, and gradient boosting) that have been effective on tabular phishing features [9], [11], [12]; and emit $p = \Pr(\text{phish} \mid \mathbf{x})$. Class imbalance, if present, should be handled with class weights or resampling rather than ignored. Accuracy is insufficient because a trivial majority classifier can look strong on skewed data, and because a false negative (phishing labeled legitimate) has a higher user-safety cost than a false positive, which mainly costs convenience [8], [11]. Precision, recall, F1-score, and ROC-AUC are therefore mandatory.

G. Explainability

Rule explanations in the prototype are the strings appended when a penalty fires. For a future model $g(\mathbf{x}) = p$, SHAP values ϕ_i satisfy

$$g(\mathbf{x}) = \phi_0 + \sum_i \phi_i, \quad (6)$$

where ϕ_0 is the baseline expectation [17]. A user-facing explanation would list the largest positive ϕ_i (evidence for phishing) and negative ϕ_i (evidence against). A valid certificate should be allowed to appear as negative evidence without being described as proof of legitimacy. SHAP is planned; no SHAP values were computed in the inspected project.

H. Safe Website Scanning and SSRF Controls

The scanner is itself a confused deputy [14]. Table II separates controls that exist in the prototype from controls that remain required before any open deployment that fetches arbitrary URLs.

TABLE II
SSRF-RELATED CONTROLS. IMPLEMENTED ITEMS WERE VERIFIED IN SOURCE. REQUIRED ITEMS ARE SPECIFIED, NOT CLAIMED AS COMPLETE.

Control	Status
HTTP/HTTPS allow-list	Implemented
Unsupported-scheme reject	Implemented
Origin-only transmission	Implemented
API body-size limit (8 kB)	Implemented
Collector timeouts (8 s)	Implemented
Reject embedded credentials	Required / planned
Resolve then deny private/loopback	Required / planned
Deny link-local and unique-local	Required / planned
Deny cloud-metadata addresses	Required / planned
DNS-rebinding check at connect	Required / planned
Redirect-hop revalidation	Required / planned
Response-size and redirect caps	Required / planned
API rate limiting	Required / planned

Implemented controls are limited to HTTP/HTTPS allow-listing, rejection of a documented unsupported-scheme family, origin-only forwarding (path and query stripped), an 8 kB JSON body limit, and 8 s collector timeouts. Required or planned controls, aligned with OWASP guidance [14], include URL normalization with rejection of embedded credentials; DNS resolution of the hostname; denial of loopback, RFC 1918 private space, link-local, multicast, IPv6 unique-local, and well-known cloud-metadata addresses [13]; defense against DNS rebinding by checking the resolved address at connect time; re-validation of every redirect hop; response-size and redirect-count limits; and API rate limiting. The prototype must not be described as SSRF-safe until those controls exist and have been tested.

VI. IMPLEMENTATION

A. Implementation Status

Table III records what the inspected repository `seethinajayadileep/WebSentinel` (commit `7d8822f`, 29 August 2026) actually contains. The workspace that hosts this paper is a different project and does not include WebSentinel application code. Capstone documentation describes a two-semester plan; later items are not claimed as built.

B. Why the Prototype Stack Was Used

The shipped prototype favors a small JavaScript surface that can be reviewed for a browser-store privacy policy. Express provides a minimal JSON API that the popup can call. `ssl-checker` isolates TLS metadata collection. `Axios` and `Cheerio` exist only to parse the WHOIS HTML provider. These choices are expedient for a first-semester demonstrator; they are not an argument against the planned Python stack.

C. Why the Proposed Stack Is Specified

FastAPI is specified for a later backend because collectors, feature extraction, and scikit-learn inference share a Python process model and async HTTP clients. PostgreSQL fits relational scan trees. Redis plus a worker is appropriate once

TABLE III
IMPLEMENTATION STATUS OF WEBSENTINEL COMPONENTS, VERIFIED
AGAINST INSPECTED SOURCE.

Component	Status
Chrome extension UI	Implemented (Manifest V3)
Express /check API	Implemented
HTTPS / TLS / WHOIS age	Implemented
Rule scoring	Implemented
Human-readable reasons	Implemented
Origin-only URL transmission	Implemented
Vercel deployment config	Implemented
RDAP client	Planned (empty file)
FastAPI / React dashboard	Planned
PostgreSQL models	Planned
Redis / Celery / RQ	Planned
DNS, redirects, technology, IP	Planned
Threat-intelligence APIs	Planned
Machine learning / SHAP	Planned
Authentication, history, reports	Planned
Monitoring / change alerts	Planned
pytest / Vitest / Playwright / ZAP	Planned (one Node assertion)
Docker / Nginx / CI	Planned

scans exceed request timeouts. React and TypeScript support a typed dashboard. Docker and Nginx support reproducible deployment. GitHub Actions would run the planned pytest, Vitest, Playwright, and ZAP jobs. pandas and scikit-learn are the conventional tabular-learning toolchain used in comparable studies [11], [12]. SHAP is specified because additive attributions map naturally onto a reason list [17].

D. Prototype Workflow

Fig. 2 summarizes the implemented path. Opening the toolbar icon queries the active tab, parses the tab URL, and either shows a local rejection or posts the origin to the check endpoint. The server re-parses the origin, runs TLS and WHOIS lookups in parallel, computes S , and returns hostname, score, message, and conditions. The popup maps the message to a visual tone and lists reasons. The popup copy states that a low score is not a guarantee of safety.

E. Scoring Pipeline

Fig. 3 shows how indicators enter S . Machine-learning fusion via (4)–(5) is drawn as a planned input.

F. Planned Learning Pipeline

Fig. 4 records the intended learning workflow. It is a design, not an executed experiment.

VII. EVALUATION PROTOCOL

This section specifies how evaluation should be conducted. It is not a report of completed experiments. Measured hardware descriptions, dataset cardinalities, classifier metrics, and scan latencies were not found in the inspected materials (no notebooks, metric dumps, or executed test suites accompanied

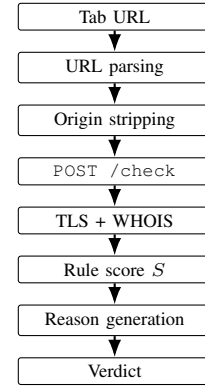


Fig. 2. Implemented prototype workflow. Path and query never leave the device.

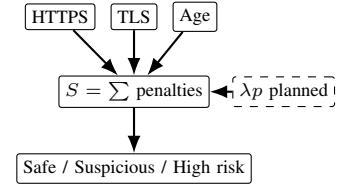


Fig. 3. Risk-score pipeline. Prototype bands are given in (3). The dashed input is not implemented.

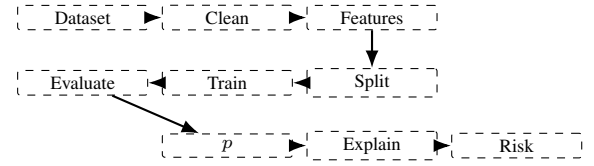


Fig. 4. Planned machine-learning pipeline. No trained weights exist in the inspected repository.

a completed study). Quantitative results are therefore not reported in this version.

A. Software Environment Under Test

The prototype runtime declared by the inspected backend manifest is Node.js with Express, axios, cheerio, and ssl-checker. The popup targets a Chromium-based browser using Manifest V3. A later evaluation of the proposed stack would additionally name Python, FastAPI, PostgreSQL, Redis, and React/TypeScript once those components exist. Hardware used for future timing runs should be recorded at execution time rather than guessed here.

B. Rule-Based Assessment Protocol

The implemented engine can be evaluated without a trained model by scoring a licensed, labelled URL set and mapping the three-level verdict in (3) onto phishing versus legitimate labels after a documented decision rule. Candidate sources include the Hannousse–Yahiouche web-page phishing set [12] or a freshly sampled PhishTank/OpenPhish corpus plus a documented legitimate crawl. Metrics shall include accuracy, precision, recall, F1-score, and a confusion matrix, with explicit review of false negatives. Class counts, collection dates,

and the train/hold-out policy must be taken from the chosen licensed source at experiment time.

C. Performance Protocol

Wall-clock latency of `POST /check` should be recorded for established HTTPS sites, HTTP-only sites, newly registered domains, invalid-TLS hosts, and timeout or DNS-failure cases. Report mean, median, minimum, maximum, timeout rate, and collector error rate. Collector timeouts of 8 s bound worst-case latency in the prototype; measured averages remain part of the planned evaluation and are therefore not reported in this version.

D. Security Protocol

Security testing should combine (i) unit tests for URL and address-policy functions, (ii) negative tests that attempt loopback, RFC 1918, link-local, unique-local, and metadata targets, (iii) redirect-to-private and DNS-rebinding cases, and (iv) an OWASP ZAP baseline against the WebSentinel application itself—not against third-party scanned sites [15]. The prototype must not be described as SSRF-safe until those tests exist.

E. Functional Protocol

A complete program would include unit, integration, API, frontend, end-to-end, security, performance, and—once a model exists—learning evaluation. The inspected backend defines a single assertion: an unsupported `chrome://` target must be rejected. Planned functional cases include valid HTTPS, HTTP-only, malformed URL, unsupported scheme, invalid TLS, newly registered domain, collector timeout, and DNS failure. Observed pass/fail counts are not reported because that suite has not been executed.

F. Learning-Evaluation Protocol

Once a model exists, evaluate logistic regression, a decision tree, random forest, and gradient boosting with cross-validation or a chronological split to reduce leakage from near-duplicate campaigns [8], [12]. Report accuracy, precision, recall, F1-score, and ROC-AUC. Analyze false negatives individually: a missed banking clone is more severe than a missed generic spam page.

VIII. SOURCE-INSPECTION OBSERVATIONS

No experimental campaign, labelled-score spreadsheet, or learning-metric file was available. This section therefore reports only qualitative observations that follow directly from source inspection. These observations are not experimental results.

The prototype returns a numeric S , a three-level message, and zero or more reason strings. Domain youth is the heaviest implemented penalty, which is consistent with literature that treats registration age as a strong external feature [7], [12]. Invalid TLS and missing HTTPS are each weighted equally, so an HTTP site with a failed certificate check can already reach the Suspicious band without being young. DNSSEC can reduce S by a small bonus that cannot cancel a new-domain

penalty. As noted in Section V-E, the current rule maximum differs from the 0–100 presentation scale.

Origin-only processing improves privacy: path, query, and fragment never leave the device. The same design, however, prevents the prototype from using lexical path features that URL-only classifiers exploit [11]. Dependence on a scraped WHOIS page is a reliability risk; RDAP would be the standards-compliant replacement [4]. The implemented reason list operationalizes the finding that users need more than a padlock [1], but the same list can over-warn on legitimate new businesses and under-warn on aged, certificated clones—a known limitation of host-feature systems [8], [16].

Claims of detection rate, latency, or test-pass percentage are unwarranted until the protocol in Section VII is executed. Empty metric templates are therefore omitted from this version; they are retained as commented source and in the accompanying evaluation-table file for later use.

IX. SECURITY AND PRIVACY CONSIDERATIONS

A. Safe Scanning

Collectors must behave as unauthenticated public clients. The design forbids exploit payloads, credential posting, authentication bypass, and authenticated crawling.

B. SSRF and Abuse

Section V-H is the primary technical control discussion. Rate limiting, authentication of the API, and refusing to become an open relay are required before wide deployment. The prototype’s open CORS policy is convenient for a demonstration and overly permissive for production.

C. Privacy and Storage Minimization

The extension’s privacy policy states that only the origin is transmitted, that path and query remain on device, and that names, passwords, payment details, accounts, and cross-tab history are not collected. That policy matches the popup implementation. Planned databases should store the assessed origin, collector outputs, and reasons—not page cookies or form bodies. A formal retention policy must be established before persistent scan storage is deployed.

D. API-Key and Third-Party Security

Future reputation or breach APIs must keep keys in server-side secrets, never in the extension. The current WHOIS HTML provider is an implicit third party and should be replaced or contractually documented.

E. Input Validation and Ethics

JSON body size is already capped at 8 kB. Further validation should constrain hostname length and character set after IDN decoding. Advisory language in the UI already declines to guarantee safety. Published papers and demonstrations should use owned or publicly documented URLs and licensed phishing datasets, not live credential-harvesting sites for interactive login tests.

X. LIMITATIONS

The following limitations are inherent or presently observed: dependence on third-party WHOIS presentation HTML and, later, on threat-intelligence quotas; incomplete RDAP coverage and redacted registration fields; false positives on young legitimate domains and false negatives on aged, well-certificated phishing; rapid campaign turnover and dataset drift [2], [12]; no JavaScript execution or DOM analysis in the prototype, so CAPTCHAs and heavily client-rendered pages will defeat naive HTML fetches; CDN and anycast hosting that obscure geolocation even when IP lookup is added; the fact that HTTPS does not imply legitimacy; the absence of a trained learning model and therefore of measured generalization or SHAP study; adversarial manipulation of lexical features and freely obtained certificates; external-API latency and timeouts; incomplete SSRF controls for resolved private addresses; and the absence of persistent history, authentication, monitoring, or a report pipeline in the inspected code.

XI. FUTURE WORK

Planned work, none of which is claimed as implemented, includes replacing WHOIS scraping with RDAP; DNS, redirect, technology, and IP/hosting collectors; approved reputation and breach APIs; a documented tabular model such as random forest or gradient boosting [9], [12]; SHAP explanations [17]; PostgreSQL-backed history; downloadable reports; authenticated users; monitoring with change alerts; stronger brand-impersonation checks, including visual or logo methods in the spirit of Phishpedia [16]; DOM and login-form analysis; NLP on URL tokens [11]; graph-based domain infrastructure features; a production SSRF resolver guard; email-gateway or browser integration; and cloud deployment with HTTPS, logs, and backups. These additions will be evaluated; they may improve detection performance, but that outcome is not assumed.

XII. CONCLUSION

Users cannot reasonably assemble protocol, certificate, and registration evidence every time they receive a link, and HTTPS does not resolve that problem. This work demonstrates the feasibility of combining publicly observable domain, protocol, and TLS indicators into an explainable website-risk assessment while explicitly considering the security risks introduced by URL-scanning services. The implemented prototype is a browser extension and REST service that returns an advisory score, a three-level verdict, and a reason list; it does not constitute a completed machine-learning phishing detector. Scanner-side SSRF remains only partly controlled. Current limitations include third-party WHOIS dependence, uncalibrated rule weights, and the absence of measured detection performance. Planned empirical validation—on a licensed labelled corpus, with the security and latency protocols in Section VII—is required before any claim of detection performance can be made.

REFERENCES

- [1] R. Dhamija, J. D. Tygar, and M. Hearst, “Why phishing works,” in *Proc. SIGCHI Conf. Human Factors Comput. Syst. (CHI)*, 2006, pp. 581–590.
- [2] Anti-Phishing Working Group, “Phishing activity trends report, 4th quarter 2024,” APWG, Tech. Rep., Mar. 2025. [Online]. Available: <https://apwg.org/trendsreports/>.
- [3] S. Marchal, J. François, R. State, and T. Engel, “PhishStorm: Detecting phishing with streaming analytics,” *IEEE Trans. Netw. Service Manag.*, vol. 11, no. 4, pp. 458–471, 2014.
- [4] A. Newton, B. Ellacott, and N. Kong, “HTTP usage in the registration data access protocol (RDAP),” IETF RFC 7480, 2015.
- [5] E. Rescorla, “The transport layer security (TLS) protocol version 1.3,” IETF RFC 8446, 2018.
- [6] Y. Zhang, J. I. Hong, and L. F. Cranor, “CANTINA: A content-based approach to detecting phishing web sites,” in *Proc. 16th Int. Conf. World Wide Web (WWW)*, 2007, pp. 639–648.
- [7] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, “Beyond blacklists: Learning to detect malicious web sites from suspicious URLs,” in *Proc. 15th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD)*, 2009, pp. 1245–1254.
- [8] G. Xiang, J. Hong, C. P. Rose, and L. Cranor, “CANTINA+: A feature-rich machine learning framework for detecting phishing web sites,” *ACM Trans. Inf. Syst. Secur.*, vol. 14, no. 2, pp. 21:1–21:28, 2011.
- [9] R. S. Rao and A. R. Pais, “Detection of phishing websites using an efficient feature-based machine learning framework,” *Neural Comput. Appl.*, vol. 31, no. 8, pp. 3851–3873, 2019.
- [10] C. Whittaker, B. Ryner, and M. Nazif, “Large-scale automatic classification of phishing pages,” in *Proc. Netw. Distrib. Syst. Security Symp. (NDSS)*, 2010.
- [11] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, “Machine learning based phishing detection from URLs,” *Expert Syst. Appl.*, vol. 117, pp. 345–357, 2019.
- [12] A. Hannousse and S. Yahiouche, “Towards benchmark datasets for machine learning based website phishing detection: An experimental study,” *Eng. Appl. Artif. Intell.*, vol. 104, p. 104347, 2021.
- [13] Y. Rekhter, B. Moskowitz, D. Karrenberg, G. J. de Groot, and E. Lear, “Address allocation for private internets,” IETF RFC 1918, 1996.
- [14] OWASP Foundation, “Server-side request forgery prevention cheat sheet,” OWASP Cheat Sheet Series, 2024, [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html.
- [15] —, “OWASP top 10:2021,” Online, 2021, [Online]. Available: <https://owasp.org/Top10/>.
- [16] Y. Lin, R. Liu, D. M. Divakaran, J. Y. Ng, Q. Z. Chan, Y. Lu, Y. Si, F. Zhang, and J. S. Dong, “Phishpedia: A hybrid deep learning based approach to visually identify phishing webpages,” in *Proc. 30th USENIX Security Symp.*, 2021, pp. 3793–3810.
- [17] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 4765–4774.